

Day2 · 7차시

에이전트란? Reasoning LLM과 자율 루프

앱을 만들었다면, 이제 스스로 일하는 앱으로.
day1에서 만든 MVP를 에이전트로 키우는 첫걸음.

SECTION 00

오프닝 — 출력이 아니라 결과

코드를 받아 붙이는 것 vs 목표를 말기고 결과를 받는 것

“코드를 받아 붙여넣는 것과, 목표를 말기고 결과를 받는 것 — 무엇이 다른가?”

— 2차시 ‘Codex에 위임’의 일반화 = 에이전트

day1 → day2

우리가 만든 앱을 다시 본다

▶ 같은 도메인, 더 똑똑하게 — 저녁 팀 프로젝트로 바로 이어진다

day1 — 앱

- 버튼을 누르면 정해진 동작
- 랜덤 추천 등 고정 로직
- 사람이 매번 조작

VS

day2 — 에이전트

- 자유로운 요청을 이해
- 스스로 판단하고 도구 사용
- 목표를 주면 결과까지

이 차시 동안 얻는 것

▶ 강의 4블록 + 실습 — 이론 절반, 손으로 절반



SECTION 01

코드 생성기 vs 에이전트

출력을 만든다 vs 결과를 만든다

코드 생성기

- 출력(텍스트) 을 만든다
- 사람이 받아 붙이고 실행
- 맥락은 매번 다시 설명
- 한 번의 응답으로 끝

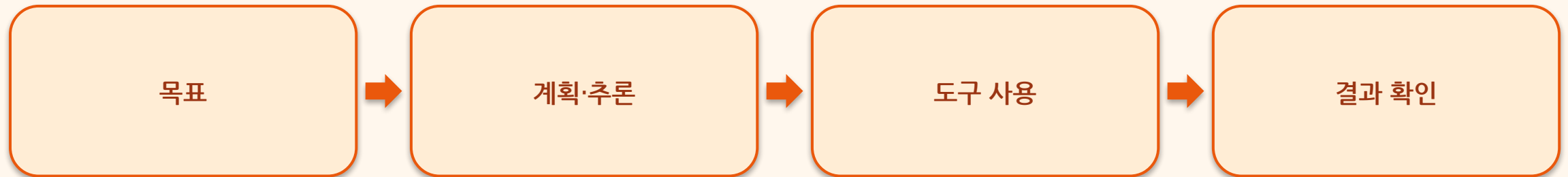
VS

에이전트

- 결과(완료된 작업) 를 만든다
- 스스로 도구 실행·검증
- 목표만 주면 경로는 알아서
- 될 때까지 반복

목표를 주면 스스로 돈다

▶ 사람은 매 단계를 운전하지 않는다 → 목표 제시 + 결과 검토



↺ 부족하면 다시 — 충분할 때까지

에이전트의 4요소

▶ 오늘은 두뇌(Reasoning)+루프. 도구·메모리·지식은 8~11차시에서

① Reasoning LLM (두뇌)

무엇을 할지
판단·추론

② 도구 (손발)

세상과 상호작용
검색·계산·API

③ 루프 (반복)

도구→관찰→
다시 추론

④ 메모리 (기억)

과거·맥락 유지

이 4요소 그림은 Day2 내내 계속 다시 나온다 — 오늘은 ①과 ③.

에이전트 해부도 — Reasoning LLM을 중심으로

▶ 책 구조 = Day2 커리큘럼 — 오늘은 가운데(두뇌)부터



에이전트가 실제로 하는 일

▶ 단발 답변을 넘어 여러 단계의 작업을 끝낸다 — 우리 MVP도 이 방향으로

코드

리포 읽고 PR까지
(2차시의 Codex)

리서치

검색→종합→리포트

운영

로그 감시→알림·조치

SECTION 02

Reasoning LLM — 생각하는 모델

즉답 vs 단계적 사고(Chain-of-Thought)

즉답

- 바로 답을 뱉음
- 간단한 변환엔 충분
- 복잡한 문제에선 자주 틀림

VS

추론(Chain-of-Thought)

- 생각의 단계를 펼침
- 중간 점검·자기수정
- 다단계 문제 정확도↑

같은 문제, 두 모델 — 즉답은 틀리고 추론은 맞힌다

▶ Q. “플라밍고 3마리 — 2마리 날아가고, 남은 1마리는 숨었다. 몇 마리가 보이냐?”

일반 LLM — 즉답



“3마리” X

훈련된 패턴으로 반사적 대답

Reasoning LLM — 단계적 사고

① 2마리가 날아갔다 → 1마리 남음

② 남은 1마리는 숨어 있다

③ 그러므로 보이는 플라밍고는 없다

“0마리” ✓ — 추론 단계(reasoning steps)가 보인다

System 1 vs System 2 — 두 가지 생각 방식

▶ Kahneman의 이중 과정 이론 — 책이 드는 대표 비유

System 1 — 빠른 직관

- 패턴에서 즉각 반응
- 힘 안 들이고 빠르다
- 익숙한 문제($2+2$)에 강함
- 일반 LLM의 즉답과 닮음

VS

System 2 — 느린 숙고

- 단계적 분석 후 답
- 느리지만 정확
- 낯선 문제(17×24)에 강함
- Reasoning LLM의 사고와 닮음

생각 토큰(thinking tokens) — 더 쓰면 더 맞힌다

▶ 같은 질문 “3+2는?” — 답만 뱉기 vs 풀이를 펼치기

토큰 1개 — 즉답

- “5”
- 연산 최소 — 생각할 시간이 없다

VS

토큰 15개 — 풀이를 펼침

- “3에 2를 더하면... 답은 5”
- 토큰 하나하나가 곧 **생각할 시간**

토큰 ↑



연산 ↑



생각할 시간 ↑



성능 ↑

Train-time → Test-time compute — 패러다임 전환

▶ 모델을 더 키우는 대신, 답할 때 더 오래 생각하게 한다

일반 LLM

사전학습 + 파인튜닝 — train-time

답변

Reasoning LLM

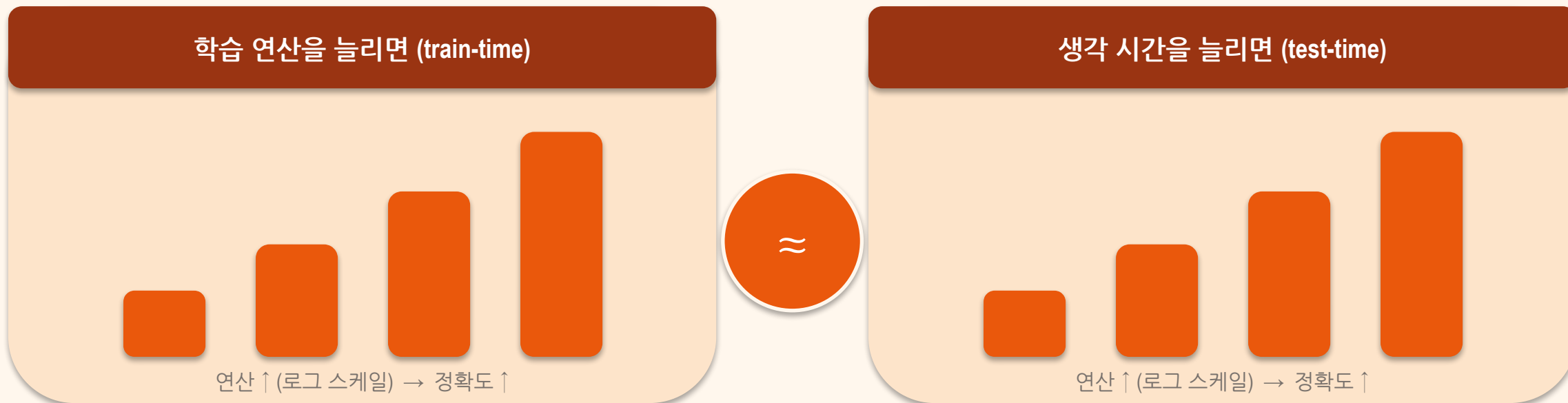
사전학습 + 파인튜닝 — train-time

답변 + 생각 (test-time compute)

학습 스케일링은 수확 체감에 부딪혔다 → 2024~ 업계는 ‘더 큰 모델’ 대신 ‘더 오래 생각하는 모델’로.

더 오래 생각할수록 정확해진다 — OpenAI o1

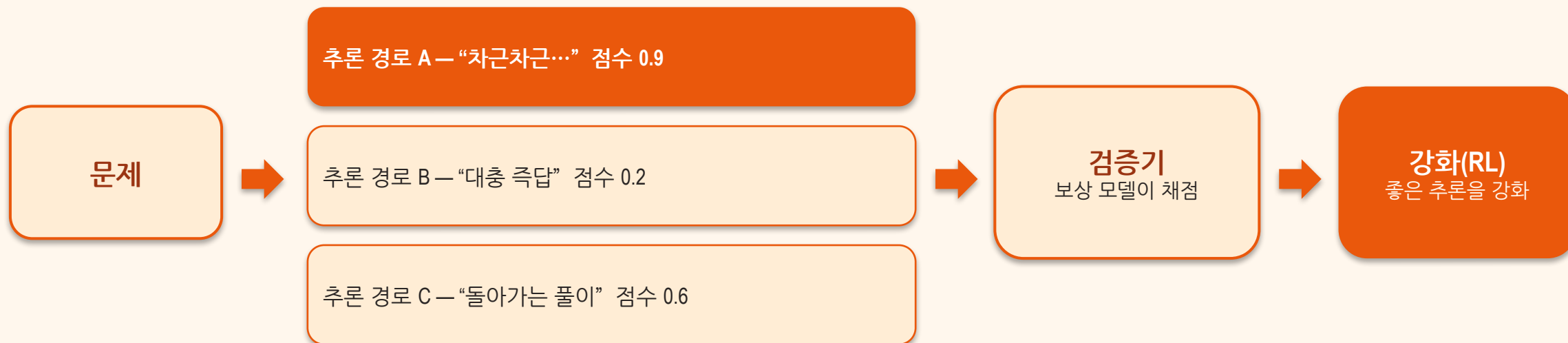
▶ 수학(AIME) 정확도 — 두 손잡이의 효과가 비슷하다



학습은 한 번뿐 — ‘생각 시간’은 문제마다 조절 가능한, 운영에서 훨씬 유연한 손잡이.

Reasoning LLM은 어떻게 학습되나

▶ 여러 추론 경로 생성 → 검증기(보상 모델) 채점 → 잘한 추론을 강화(RL)



반복할수록 '생각하는 법'이 학습된다 — DeepSeek-R1: 정답 보상만의 순수 RL로 추론 능력 발현

Reasoning 모델 vs 일반 모델

▶ 추론력·비용·속도는 삼각 트레이드오프 — 셋 다 가질 수 없다

일반 모델

- 빠르고 저렴
- 단순·대량 작업
- 요약·분류·변환

VS

Reasoning 모델

- 느리고 비쌈 (토큰을 더 씬)
- 다단계 계획·도구 사용
- 복잡한 의사결정

왜 중요한가

Reasoning이 루프의 엔진이다

▶ 매 스텝의 판단이 곧 추론 — 추론이 약하면 루프가 헤맨다

다음 행동 선택
무엇을 할까

도구·인자 결정
어떤 도구로

관찰 결과 해석
무엇을 알았나

계속/종료 판단
충분한가

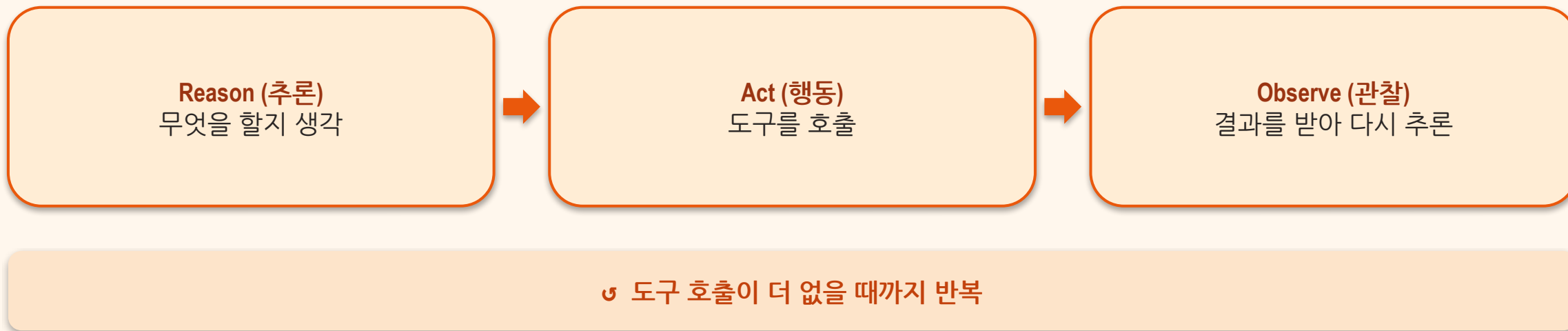
Reasoning = 루프의 엔진 — 에이전트의 모든 의사결정

SECTION 03

ReAct 루프 — 추론하고, 행동하라

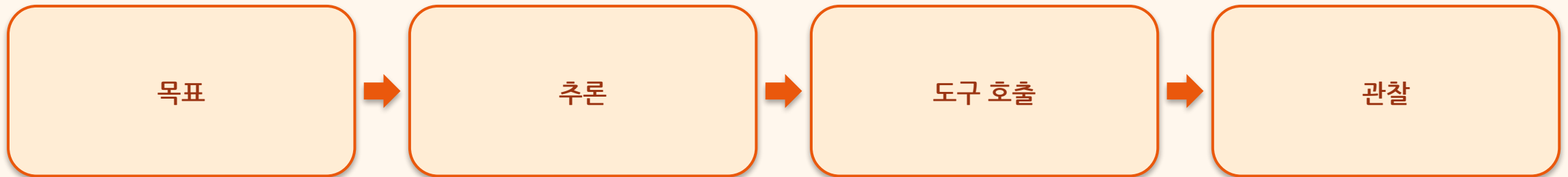
Reason + Act + Observe

▶ 2차시 Codex가 파일을 읽고 고치고 테스트하던 그 루프와 같은 뼈대



에이전트 루프 한눈에

▶ 한 턴 안에 추론·도구 호출이 수십 번 일어날 수 있다



🔄 더 필요하면 반복 → 종료는 '최종 답'

도구 호출 한 조각

▶ 모델이 '이 도구를, 이 인자로' 구조화 요청 → 우리가 실행 → 결과를 다시 모델에

```
tools = [{"type": "function", "function": {  
    "name": "search_restaurants",  
    "parameters": {...}}}]  
# 모델 → search_restaurants(area="강남", taste="매움")  
# 우리 → 실행 결과(JSON)를 messages에 다시 추가 → 반복
```

종료 조건 & 흔한 실패 3가지

▶ 무한 루프·헛 도구·관찰 무시 — 실습에서 그대로 겪게 될 것들

종료 조건

더 호출할 도구가 없고 정보가 충분하면 → 최종 답 생성

실패① 무한 루프

같은 도구만 반복 호출 → **max_steps** 상한으로 차단

실패② 잘못된 도구

설명이 모호하면 엉뚱한 도구를 부름 → 설명을 명확히, 개수를 줄여라

실패③ 관찰 무시

도구 결과를 messages에 안 넣으면 모델은 아무것도 못 본다

SECTION 04

에이전트 유형 & 모델 선택

얼마나 자율적이어야 하나

▶ 자율성↑ = 유연성↑·예측가능성↓ — 막연하면 워크플로로 좁혀라

고정 워크플로

예측 가능 — 정해진 순서 (자율성 최저)

도구 쓰는 LLM

한두 도구 + 단순 루프 — 챗봇·RAG가 여기

자율 에이전트

개방형 — 스스로 계획·반복 (자율성 최고)

실무의 에이전트 7가지 (building ch.1)

▶ 우리 팀 MVP는 어디에 가까운가?

① 업무 자동화

정해진 업무 흐름 대행 — 폼 처리·리포트 발송 (자율성 낮음)

② 대화형

고객 문의를 이해하고 답·처리 — 챗봇·고객센터

③ 리서치

검색 → 수집 → 종합 리포트 작성

④ 분석

정형 데이터 조회·분석 — “지난달 매출은?”

⑤ 개발자

코드 생성·리팩터링·PR — 2차시의 Codex

⑥ 도메인 특화

법률·의료 등 전문 분야 판단 보조

⑦ 브라우저 사용

사람처럼 웹을 열고 클릭·입력해 작업 수행

작업에 맞는 모델

▶ 원칙: 최신 범용 모델로 단순하게 시작 → 규모·비용 제약이 생기면 라우팅

빠른 모델

- 단순·대량·실시간
- 분류·요약·추출
- 비용 절감

VS

Reasoning 모델

- 복잡·다단계
- 계획·도구 오케스트레이션
- 정확도 우선

에이전트 vs 워크플로, 어떻게 정하나

▶ 팀 프로젝트에서 바로 쓸 4문항 — 자율성은 필요한 만큼만

1

작업이 매번 같은 순서인가?

그렇다 → 워크플로(또는 단순 함수)

2

입력이 열려 있고 경로가 다양한가?

그렇다 → 에이전트(도구+루프)

3

실패 비용이 큰가?

사람 승인(Human-in-the-loop) 추가

4

도구가 꼭 필요한가?

아니면 단발 LLM 호출로 충분

SECTION 05

[실습] 내 MVP를 에이전트로

day1에서 만든 앱 → ReAct 에이전트 (공통 예시: 맛집 추천 도우미)

무엇을 하나 (약 20분)

- 실습/7차시_실습.ipynb — day1 추천 앱을 에이전트로 발전
- 공통 예시 ‘맛집 추천 도우미’로 ReAct를 익히고
- 마지막에 내 팀 MVP의 도구로 바꿔 적용
- 코드는 위임·검토, 우리는 루프 흐름을 눈으로 확인

4단계로

▶ ③까지 공통, ④는 팀별 — 체크포인트: 도구 호출 로그가 찍히는가

1

도구 정의

내 앱의 기능을 function tool로 (검색·계산)

2

ReAct 루프 실행

자유 요청 → 추론·도구·관찰 단계 출력 확인

3

iterate

요청을 바꿔가며(작게 자주) 동작 확인

4

내 MVP에 적용

우리 서비스 도구 2개로 교체 → ‘내 MVP 에이전트’

도구 정의 (맛집 도우미 예)

▶ 이 '설명'이 모델이 도구를 고르는 유일한 단서

```
def search_restaurants(area, taste=""):
    ... # day1 앱의 DB/API 자리
TOOLS = [{"type": "function", "function": {
    "name": "search_restaurants",
    "parameters": {"type": "object",
        "properties": {"area": {"type": "string"}}}}}]
```

우리 서비스로 바꾸려면

▶ 루프는 그대로, 도구만 교체 — 이게 저녁 팀 프로젝트의 첫 커밋

- MVP의 핵심 기능 2개를 도구로 노출 (예: 일정앱→add_event, 가계부→add_expense)
- 시스템 프롬프트에 에이전트의 역할·범위를 한 줄
- 나머지(루프)는 그대로 — 도구만 바꾸면 끝

```
codex "우리 MVP 기능 2개를 function tool로 정의해 ReAct에 연결해줘"
```

SECTION 06

정리

오늘 3줄

- 에이전트 = Reasoning LLM(두뇌) + 도구(손발) + 루프 — 출력이 아니라 결과
- Reasoning이 루프의 엔진 — 더 오래 생각(test-time compute)할수록 정확
- 자율성은 필요한 만큼만 — 막연하면 워크플로로 좁혀라

AI는 도구, 결정과 책임은 사람.

— 에이전트가 빨라질수록 — 승인 전 검토는 더 신중하게

다음 차시

Day2 로드맵에서 지금

▶ 다음 8차시 — 무엇을 맡기고(스코핑) 어떻게 신뢰를 주나



14 [캡스톤] AI 마이노트 개발기 — 오늘 만든 에이전트를 매 차시 키워간다